Smile: a first Java program

Learn the basic structure of a Java program, while writing code to draw a smiley face.

WE'LL COVER THE FOLLOWING

- A first program: smiley

Here is a Java program to draw a circle. We'll dissect it in a minute. First, run it by clicking on the *Run* button below the code.

```
// include educative's simple graphics library:
import com.educative.graphics.*;

class DrawCircle {
  public static void main(String[] args) {
    // set up a canvas for drawing:
    Canvas c;
    c = new Canvas(200, 200);

    // call some graphics commands:
    c.fill("yellow");
    c.stroke("black");
    c.circle(100, 100, 50);

    // nothing is actually drawn on the screen until
    // the `draw` method is called:
    c.draw();

  }
}
```

First things to note:

1. Single-line comments begin with `//`.
2. All functions in Java are called *methods*. `c.circle()` calls the method `circle` with the parameters `100`, `50`, `50`, specifying x and y coordinates for the center, and the radius.

3. Some method calls need an object. `circle` needs a Canvas object to draw with. The first few lines set up a reference to a Canvas object in the variable `c`. Then the `c.circle()` method call acts on that canvas object.
4. The method `main` is *defined* using the keywords `public static void`. The method named `main` is special: Java starts running the code at the first line of the method named `main`.
5. Method definitions are grouped into *classes*.
6. Most lines of code end in a semi-colon. Method and class definitions do not.

A first program: smiley

As a warmup, write a program in the next code box that causes a yellow smiley face to be drawn on the screen. You have a 200 by 200 window available.

1. Draw the outline of the face after the comment `// draw the outline of the face`. Don't forget the semi-colons.

2. Draw the eyes. Put your code for drawing the eyes after the line `// draw the eyes`.

3. Bonus challenge: draw the mouth. Hint – draw a circle, and then erase the top part of it by drawing a yellow rectangle that covers the top of the mouth but fits within the face. (Experiment with `c.rect`, which takes four parameters.)

You can test your code with the *Run* button, and click on the second tab to compare to a sample solution when you are done.

Smiley | Sample solution

```
import com.educative.graphics.*;

class Smiley {
  public static void main(String[] args) {
    Canvas c;
    c = new Canvas(200, 200);

    // Draw the outline of the face
```

```
    c.fill("yellow");
    c.stroke("black");
    c.circle(100, 100, 50);


    // draw the mouth

    // draw the eyes

    c.draw();
  }
}
```

Writing your own static methods

Learn to write static methods, which play a similar role to ordinary functions in other languages.

WE'LL COVER THE FOLLOWING

- Exercise: factor smiley
- Check your work

The main unit of organization in a Java program is the **class**. The simplest Java program contains just one class. Among other things, a class is a collection of methods.

There are two types of methods: **static** methods and **non-static** methods. We will focus on static methods first, since they are simpler. A static method plays a similar role to an ordinary function in other languages.

In any language, *factoring* is the process of reorganizing code into different files, classes, methods, libraries, or functions. You will factor some code into new static methods in the next exercise.

Exercise: factor smiley

The code below has a single static method, called `main`. The method is declared using the keywords `public static void`, the name of the function, parentheses containing parameters for the function (`String[] args`), an open curly brace, and a closing curly brace.

1. Write three static methods: `drawOutline`, `drawMouth`, and `drawEyes`. These methods should take no parameters, and you can use the keywords `public static void` to define them. If you usually code in Python, don't forget the curly braces.

2. Factor the drawing code, except `c.draw()`, into these functions. Leave the

code `c = new Canvas(200, 200);` in `main`. The variable `c` will be available to all three of these functions, since it is declared in the class, outside of any function. This is necessary so that you can make calls to `c.circle`, etc.

3. Call your functions from `main` using `drawOutline()`, etc., to draw the smiley. (Static methods within the same class can be called just like functions in other languages, so no dot is required.)

SmileyFactored.java | SmileyFactoredSolution.java

```
import com.educative.graphics.*;

class SmileyFactored {
  static Canvas c;
  public static void main(String[] args) {

    c = new Canvas(200, 200);

    // Draw the outline of the face
    c.fill("yellow");
    c.stroke("black");
    c.circle(100, 100, 50);

    // draw the mouth
    c.stroke("black");
    c.fill("yellow");
    c.circle(100, 100, 30);
    c.stroke("yellow");
    c.rect(68, 68, 62, 40);

    // draw the eyes
    c.stroke("black");
    c.fill("black");
    c.circle(100 - 20, 100 - 10, 5);
    c.circle(100 + 20, 100 - 10, 5);

    c.draw();
  }
}
```

Check your work

There is a sample solution in the tab labelled **SmileyFactoredSolution.java** above. Compare it to your solution. Here are a few things to look for:

1. Coding style: comments. Do each of your methods have a comment at the top that describes the intent of the method?

top that describes the intent of the method.

2. Coding style: indentation. Every method declaration in a class should be indented one level. Then, the body of the method should be indented another level as well.
3. Coding style: blank lines. Are there blank lines between your method definitions? Do blank lines separate logical groupings in the main method?
4. Coding style: make sure there is space around operators and after commas.
5. All of your methods should be declared using `public static void`. We'll see shortly what these keywords mean.

Printing to the screen

Use standard method calls to print to the screen.

WE'LL COVER THE FOLLOWING

- Exercise: Beatles
- String concatenation
- Special characters: printing quotes, newlines, tabs, etc.
  - Exercise: translate from Python
- Formatted printing with `format`

`print` commands aren't the most exciting thing in coding, but eventually, you'll want to print something out, even if only for debugging or testing your code. You won't find anything too surprising in Java's methods for printing, but there are some slight differences from C, Python, and Javascript.

You can print the text "Hello, World!" to the screen using a the method call `System.out.println("Hello, World!");`.

Why so much typing for a simple print? Frequently, method calls require an **object** to act on in Java. In the method call `c.circle(150, 100, 25);` from the first section, the method `circle` is called to draw a circle, acting on the object referenced by `c`, a `Canvas` object that we first had to create.

`System.out` is also the name of an object: the main printed output device for your program. So `System.out.println` calls the method `println` on the `System.out` object.

`println` requires one parameter, which should be a string of text, like `"Hello, World"`, or something that Java can convert to a string, like `6`. Strings are marked by double quotes `"` in Java.

Exercise: Beatles

As practice writing a complete program, write a program that prints out the text:

```
Hello, hello.
I don't know why you say goodbye.
I say hello.
```

You'll need a class name that matches the name of the file, a `main` function (don't forget the `public static void` or the required parameters), and a few `println` calls. Run your code to make sure it works, and then take a look at the solution in the second tab.

Beatles.java | BeatlesSolution.java

note: The `System.out.println` method prints a newline after the text. Use `System.out.print` if you don't want the newline.

String concatenation

Strings can be concatenated in Java using the `+` operator. `System.out.println("My name is " + "Inigo Montoya")` first concatenates the two strings, and then prints the result. As long as one of the items is a string, the two items are converted to strings before concatenation. So

```
System.out.println("My favorite number is " + 42);
```

converts `42` to a string, concatenates, and prints. Order of operations matters. What would the following print?

```
System.out.println(42 + 22 + " is my favorite number.");
```

Special characters: printing quotes, newlines, tabs, etc.

Just like in Python, Javascript, or any other language influenced by C, you can

use backslash codes. `\n` will give you a newline, and `\t` will give you a tab.

In some languages, like Python, you can use either single, double, or triple quotes to create a string, and a clever choice will allow you to print quotes of a different type within a string. Java doesn't have this capability, so if you want to print double quotes, you'll need the code `\"`.

Exercise: translate from Python

The first tab below has some Python code that prints a quotation from a well-known novel. Run it. In the second tab, write a Java program that prints the same output. It's ok if you don't know Python – you should be able to see the intent of the code by running it. A few things to note:

1. Every Java program needs a class and a `main` method.
2. Python concatenates adjacent string literals. Java does not. If you want to split the long line into multiple lines, you'll need to concatenate several strings, as done in the sample solution.
3. Python's `print` statement automatically prints a newline; `System.out.println` is the equivalent.

ged.py | Ged.java | GedSolution.java

```
# Python code to print some dialog from a novel.
print('"Good," said the boy, for he had no wish '
      'to tell the secret to his playmates, '
      'liking to know and do what they knew not and could not.')
```

Formatted printing with `format`

Java provides a method `format` that works like the `printf` function in `C`, or like the string substitution operator `%` in Python. If you know how one of those works, just skip to the example at the end of this section. Otherwise, read on.

Sometimes, you'd like to print out a combination of text data and values, but you'd like to keep your code clean and readable without a lot of string

concatenation. For example, maybe you'd like to print out the value of π. The

`format` method will let you use a *format string* as the first parameter, and substitute later parameters into that string.

`System.out.format("%f is an approximation of pi.", 3.14159)`.

The `%f` is a *format specifier* marks the location where the parameter `3.14159` should be substituted. The letter `f` in `%f` indicates that the value will be a floating point number. For an integer, use `%d`.

Sometimes you'd like your output to be formatted nicely. You can do things like round the number as it is inserted, or pad the number with spaces. A format specifier like `%9.2f` would print a floating point number with 2 digits after the decimal, left-padded with spaces to take up 9 characters (including the decimal). A few examples:

printExamples.java

```
class printExamples {
  public static void main(String args[]) {
    System.out.format("My favorite number is %.3f.\n", 3.141592654);

    // part of a times table:
    System.out.format("%2d %2d %2d\n", 4, 6, 8);
    System.out.format("%2d %2d %2d\n", 8, 12, 16);
  }
}
```

Variables; integer values with `int`

Java is a statically-typed language, which means that you must explicitly create each variable and tell what type of data (for example, an integer) that variable will store before using it.

WE'LL COVER THE FOLLOWING

- Variable declaration
- Java variable naming convention: camel case
- Combined declaration and assignment

Here is an example of creating and using a variable that will store an integer value:

```
class FortyTwo {
  public static void main(String args[]) {
    int meaningOfLife;
    meaningOfLife = 42;
    System.out.println(meaningOfLife);
  }
}
```

Variable declaration

The statement `int meaningOfLife` is called a **variable declaration** and it causes Java to reserve a space in memory to hold an integer value; it names that space `meaningOfLife`. In languages like Python or Javascript, you can skip this step, but C and C++ also require variable declarations.

The structure of a variable declaration is the type of the variable (`int` in this case) followed by the name of the variable (`meaningOfLife`).

Unlike in dynamically-typed languages like Python or Javascript, the variable `meaningOfLife` must hold an `int` now and forever. Assigning a floating-point

`meaningOfLife` must hold an `int` now and forever. Assigning a floating point value like `3.14` to `meaningOfLife` will fail with a compiler error. This may seem inconvenient, but has an advantage: if you stated the intent that `meaningOfLife` be an integer, and later try to store something else in that variable, there is an inconsistency between intent or action, a likely bug. It's better to catch such bugs at compile-time rather than during deployment.

Java variable naming convention: camel case

Variable names are often composed of many words. In Java, the first letter should be lowercase, and each new word should be uppercase. This convention is sometimes called "camel case", since the capital letters look like the humps on a camel's back: `meaningOfLife`.

In contrast, some other languages, such as Python, conventionally use separate words in variable names by underscores: `meaning_of_life`.

Combined declaration and assignment

In order to use a variable in Java, you must first do two things:

1. declare the variable, giving it a name and type
2. assign an initial value to that variable, using the assignment operator `=`

It's good to think of these as separate steps, but it is sometime convenient to write one line of code to do both at the same time, like this:

```
class CombinedDeclarationAndAssignment {
  public static void main(String args[]) {
    int x = 5;
    System.out.println(x);
  }
}
```

Fractional values of numbers with `double`

Use a double to store a number that has values after the decimal point.

WE'LL COVER THE FOLLOWING

- Exercise: circle area
- Exercise: big smile

The `int` data type only represents numbers without a fractional part. What if we want to store an approximation for π in a mathematics program? We can use a different data type, called a `double`, to store numbers that have values after the decimal point. `double` is short for *double-precision floating point*. Floating point means that there is a decimal point that can be placed at different locations (or float), in the number.

Computer programming languages evolve over time; double-precision just means that this representation allows double the precision that an earlier representation of floating-point numbers used. There is also a floating point type called `float` in Java, which uses less memory and has less precision than a double. It is rarely used.

Exercise: circle area

Objective: Create and use values and variables of the `double` type.

Declare and give initial values to variables `r` and `pi` representing the radius of a circle, and an approximation of the mathematical constant π. Use those values to compute and print the area of the circle. You can square a number in Java by multiplying it by itself.

CircleArea.java | Sample Solution

```
class CircleArea {
  public static void main(String args[]) {
```

```
        double area;
  }
}
```

Notice from the sample solution that you can declare multiple variables of the same type using a single line of code.

Exercise: big smile

Objective: use variables to allow the behavior of code to be changed easily.

The code below will draw a smiley face on the screen, and you can use the variables `x` and `y` to change where the smiley face is drawn. But the smiley face is always the same size. Add a new variable, `scale`, that allows you to change the size of the smiley face to make the face either larger or smaller. For example, if `scale` had the value `2`, then the code would draw the smiley face twice as large (but still centered on `x` and `y`).

Any time you create a variable, you should first ask what type of data it will hold, since you will need that information in order to declare the variable. Should `scale` be an integer? No, not if you'd like it to hold a value like `0.5` to make the smiley smaller.

Test your code by changing variable values a few times to draw the smiley face at different locations at both small and large scales.

Hint: Some values should be scaled; others should not be. For example, if the *x* location of the left eye is `x - 20`, then the `20` should be scaled, but the `x` should not be, since `x` will still be the x location of the center of the entire smiley, even if the smiley is scaled.

BigSmile.java | Sample solution

```
import com.educative.graphics.*;

class BigSmile {
  public static void main(String[] args) {
          Canvas c;
    c = new Canvas(200, 200);

    int x;
    int y;
```

```
    x = 100;
    y = 100;

    // Draw the outline of the face
    c.fill("yellow");
    c.stroke("black");
    c.circle(x, y, 50);

    // draw the mouth
    c.stroke("black");
    c.fill("yellow");
    c.circle(x, y, 30);
    c.stroke("yellow");
    c.rect(x - 32, y - 32, 62, 40);

    // draw the eyes
    c.stroke("black");
    c.fill("black");
    c.circle(x - 20, y - 10, 5);
    c.circle(x + 20, y - 10, 5);

  }
}
```

Arithmetic expressions and operators

Learn how to compute expressions in Java, and how to convert between different types of data using casting. Be careful of the difference between integer and floating-point division!

WE'LL COVER THE FOLLOWING

- Integer and floating point division
- Typecasting operators
- Exercise: casting
- Implicit vs. explicit casting
- Combined assignment and arithmetic operators
- Increment operators `--` and `++`

Computing values in Java works much like you'd expect from most other languages:

```
class ExpressionOperand {
        public static void main(String args[]) {
    int x;
    x = (3 * 6) + 24;
    System.out.println(x);
  }
}
```

There are two things to be aware of:

1. Like in C or C++, but unlike Javascript or Python3, division of two integers yields an integer.

2. Operators given two different types (like `2 + 1.7`) promote, or automatically convert, the more limited type. `2` will be converted to the floating point `2.0`, and the resulting value will be a floating point that

must be stored in a variable of type `float` or `double`.

Integer and floating point division

There are two types of division, and each is sometimes useful. **Integer division** takes two *integers* and evaluates to an integer. **Floating-point** division takes two *floating-point numbers* (numbers with a decimal point) and evaluates to a floating-point number.

Let's say I have 18 cents and 5 nieces. How much money should I give to each? With floating point division, 18/5 = 3.6, but it's hard to distribute .6 cents in cash, and my nieces prefer hard currency. Integer division would give the value 3. If both operands of `/` are integers, Java uses integer division. If either operand is a floating point value, then Java uses floating point division.

You can ensure floating point division by adding a decimal point and a zero:

```
class DivisionTypes {
  public static void main(String args[]) {
    System.out.println(18 / 5);
    System.out.println(18.0 / 5.0);
  }
}
```

If you want the remainder that would be left over after integer division, you can use the **modulus operator** `%`:

Modulus.java

```
class Modulus {
  public static void main(String args[]) {
    // Three cents left over after distributing
    // three pennies to each of my five nieces:
    System.out.println(18 % 5);
  }
}
```

Typecasting operators

Sometimes, you need to force Java to convert one type of data to another. For example, if you have two `int` variables and would like to divide them to get a fractional number, you might convert those values into `double` values. You can do this by typecasting the values, or **casting**, for short. A cast operator is written using parentheses and the name of the target type, and precedes the value to be cast. For example, `(double) 5` yields the value `5.0`.

For now, we'll mostly cast between numerical types. Unlike in Javascript or Python, you cannot cast an `int` or `double` to or from a string, but must rather use special methods of the `String`, `Integer`, or `Double` class.

Exercise: casting

Use casting to print out the floating point result of dividing the value in the variable `numerator` by `denominator` in the code below.

Cast.java | Sample solution

```
class Cast {
  public static void main(String args[]) {
    int numerator = 18;
    int denominator = 5;

  }
}
```

Implicit vs. explicit casting

Sometimes, Java does casting for you automatically. For example, if you write `18.0 / 5`, then Java will decide to cast `5` into a floating point `5.0`, an *implicit* cast. Another example is `double x = 5`. The `5` on the right is an integer data type, but Java knows to cast it into a double.

On the other hand, `int x = 5.6` will not work. Java will notice that you are trying to put a floating point into an `int`, and warn you that this may entail a loss of precision. You can reassure Java by using a casting operator: `int x = (int) 5.6`. Notice that when casting to an integer, the part after the decimal is

truncated, not rounded, so this expression will compute the value 5, not 6.

Combined assignment and arithmetic operators

Like many other languages, Java permits arithmetic and assignment to be combined, with the operators `+=`, `-=`, `/=`, `*=`. Thus, `x += 4` would increase the value of `x` by `4`.

Increment operators `--` and `++`

Java, Javascript, C, and C++ all have increment operators that work the same way; Python does not. The `++` operator adds one to a value, `--` subtracts. These operators are very useful in Java `for` and `while` loops, which we will see soon.

These operators are most cleanly used like the combined assignment operators in Python: you can replace the line of code `x += 1` with `x++`.

Increment.java

```
class Increment {
  public static void main(String[] args) {
    int x;

    x = 5;
    x += 1;
    System.out.println("x = " + x);

    x++;
    System.out.println("x = " + x);
  }
}
```

Variable assignments like `x = 4` change the value of a variable. In C and Java, these assignments also produce a value that can be used. `x = 4` produces the value `4`. So `y = (x = 4)`. would set `y` to have the value `4`. (The parentheses are not needed, but clarify what is happening.)

Similarly, `x++` produces a value: the value of `x` before adding `1` to `x`. This is called the **post-increment operator**, since the value is produced, and then the variable is incremented.

On the other hand, `++x` increments the variable first, and then produces the value: the **pre-increment operator**.

Overly-clever use of `++` and `--` operators are a common source of hard-to-find errors. I almost always write `x++` on its own line of code, and never use `++x`.

Just because you can write hard-to-read code like `x += ( y-- - (x = 4) )` doesn't mean you should.

Preincrement.java

```
class Preincrement {
  public static void main(String[] args) {
    int x = 5;
    int y = 5;

    // The ++ operator changes the value
    //  of the variable, but you can also
    //  use the result in an expression:

    System.out.println(x++);  // post-increment
    System.out.println(++y);  // pre-increment

    // assignment statements are also expressions.
    //  However, the code below is bad: it's too likely
    //  to be a typo; probably you wanted ==, the
    //  the equality comparison.
    System.out.println(x = y);

    // Truly horrible programming style:
    System.out.println(x += (y-- - (x = 4) ));
  }
}
```

Method parameters and return values

Learn how to define methods in Java, with typed method parameters and typed return values.

WE'LL COVER THE FOLLOWING

- Method parameters
- Exercise: smile method
- Method overloading and method signatures
- Method return values
- Exercise: circle area method
- The `main` method returns `void`
- Exiting a program with `System.exit()`

Like C, Python, and Javascript functions, a method may take parameters, and may return a single value. Of course, that single value may be a reference to a list, an object, or an array that contains other values. All value in Java must have a type, and method declarations must indicate the type of parameters and the return value.

Method parameters

Here is an example of the syntax to declare a method that takes parameters:

```
class MethodParameters {

  public static void printMyNumber(int x) {
    System.out.println("My favorite number is " + x + ".");
  }

  public static void main(String[] args) {
    printMyNumber(42);

  }
}
```

Parameters are local variables of the method. In the definition of `printMyNumber(int x)`, the code `int x` declares a new int variable, `x`. When the method is called, the value 42 is copied into the variable `x`, and then the code of `printMyNumber` is executed.

What happens if you call `printMyNumber` with a double value? The compiler will notice that you are passing in a double, and that the precision of the variable `x` is int. The compiler will give you an error message and halt. If you want to round a double down and pass it in, you are welcome to use the `(int)` cast: an example of explicit casting.

On the other hand, if you have a method that expects a double, you can pass in an int. Java will notice and do the conversion for you automatically: an implicit cast. For example:

```
class CastParameter {
  public static void main(String[] args) {
    // This works, since the int 42 can be cast
    //  implicitly into a double:
    double x = Math.sqrt(42);

    System.out.println(x);
  }

}
```

Exercise: smile method

The following code draws a single smile. Create a new method, drawSmile, that draws a smile on a specified canvas object, at a specified x and y location, at a specified scale. You'll need four parameters for drawSmile: a Canvas variable, two ints, and a double. You can declare drawSmile using the keywords `public static void`.

Test your method by calling it a few times from `main`. Your `main` method will need to create the canvas, and pass that canvas as well as other parameters to `drawSmile`.

Smile.java | Sample solution

```
import com.educative.graphics.*;

class Smile {
  public static void main(String[] args) {
    Canvas c;
    c = new Canvas(200, 200);

    int x;
    int y;
    double scale;

    x = 100;
    y = 100;
    scale = .5;

    // Draw the outline of the face
    c.fill("yellow");
    c.stroke("black");
    c.circle(x, y, 50 * scale);

    // draw the mouth
    c.stroke("black");
    c.fill("yellow");
    c.circle(x, y, 30 * scale);
    c.stroke("yellow");
    c.rect(x - 32 * scale, y - 32 * scale, 62 * scale, 40 * scale);

    // draw the eyes
    c.stroke("black");
    c.fill("black");
    c.circle(x - 20 * scale, y - 10 * scale, 5 * scale);
    c.circle(x + 20 * scale, y - 10 * scale, 5 * scale);
  }
}
```

Method overloading and method signatures

Sometimes, you want a method that works for different types of parameters. Possibly, you'd even like the method to work slightly differently for different types of parameters.

In Java, you can write two different methods with the same name in the same class, as long as the methods take different parameter types. Here's an example:

```
class Overload {
```

```
  public static void printMyFavoriteNumber(int x) {
    System.out.println("My favorite number is the integer " + x);
  }


  public static void printMyFavoriteNumber(double x) {
    System.out.println("My favorite number is the double " + x);
  }

  public static void main(String[] args) {
    printMyFavoriteNumber(42);
    printMyFavoriteNumber(3.1415);
  }

}
```

The name of a method, together with the types of its parameters in order, is called the **method signature**.

The names of the parameters are not part of the method signature, so you cannot define two methods with the same name that take the same types of parameters, even if the parameters have different names. Why? When the method is called, Java knows the types of the arguments being passed in, and can use that to determine which of the overloaded methods you intended. But Java can't tell the names of the intended formal parameters from argument values or types.

For similar reasons, the return type of the method, which we will see next, cannot be used for overloading.

Method return values

Unlike Python or Javascript, the type of value that a method returns must be stated in the method declaration. If you later try to return some other type of value, the Java compiler will indicate an invalid return type. This is very useful: coders who use the method know exactly what type of return value to expect, and that expectation can be enforced by the compiler.

An example:

```
class ReturnExample {
  public static double square(double x) {
    return x * x;
  }
```

```
  public static boolean isEven(int x) {
    return x % 2 == 0;

  }

  public static void main(String[] args) {
    System.out.println(isEven(6));
    System.out.println(square(6));
  }
}
```

In the method declaration, the last word before the name of the method indicates the type of the return value. So we can see that `square` returns a double, `isEven` returns a boolean.

Every method must have its return type declared, even if the method has no return value. A function for drawing a smile, for example, might not return anything at all, so it should have a return type of `void`.

Exercise: circle area method

Write a method that takes the radius of a circle as input and returns the area of that circle. You may use the built-in constant `Math.PI` for the value of π.

CircleArea.java | Sample solution

```
class CircleArea {

  public static void main(String[] args) {
    System.out.println("A circle of radius two has area " + circleArea(2));

  }
}
```

The `main` method returns `void`

The method `main` is declared with `public static void`. The `void` indicates that `main` does not return a value. In C or C++, the main method traditionally returns an integer, indicating to the operating system whether the program ran successfully or not.

Exiting a program with `System.exit()`

Java can exit prematurely and indicate success or failure to the operating system using `System.exit()`. Even if you don't care about the resulting value, `System.exit()` is occasionally useful for debugging, or for terminating a program early for other reasons.

```
class HardStop {
  public static void main(String[] args) {
    System.out.println("Do or do not.");

    // force quit the program, making the code
    //  0 available to the operating system.
    //  (0 traditionally indicates success.)
    System.exit(0);

    System.out.println("There is no try.");
  }

}
```

Text values with String and char

Learn to work with text data in Java.

WE'LL COVER THE FOLLOWING

- `String` is a class
  - The static method `String.valueOf()`
- Strings are objects
- Static vs non-static methods of the String class
- The `char` data type
- Strings are immutable in Java

A variable used to hold a string value in Java has the type `String`. The string data itself, when typed in quotes in code, is called a *string literal*; Java should interpret it as literal string data, and not as code.

What does the following code print? Is there an error in the code? Fix it.

Fixme.java

```
class Fixme {
  public static void main(String args[]) {
    String greeting;
    greeting = "hello";
    System.out.println(greeting);
    System.out.println(hello);
    System.out.println("greeting");
    System.out.println("hello");
  }
}
```

`String` is a class

Each of the Java programs you have seen so far was defined as a single class containing some methods. **A class in Java has two purposes:**

1. A class is a collection of methods that you may call.
2. A class defines a custom data type, and can be used to make objects of that class.

The `String` class serves both purposes: it provides some methods that are useful for `Strings`, and it defines a `String` data type that you can use to make string objects.

Let's look at the first use first. The `String` class is contained somewhere in a Java file called `String.java`. It contains within it several static methods. You can call a static method of a class by using the format `Classname.methodname()`.

Usually, before using the static methods of a class, you need to **import** that class, which we will see how to do later, but `String` is special and built-in.

The static method `String.valueOf()`

As an example of a static method in the `String` class, let's look at the method `valueOf`. It takes a single parameter, which may be a double, int, or boolean, for example. It then constructs and returns a string representation of that parameter. Let's do it:

```
class StringExampleOne {
  public static void main(String args[]) {
    String myFavoriteNumber;
    int x = 42;

    myFavoriteNumber = String.valueOf(x);

    System.out.println("My favorite number is " + myFavoriteNumber + ".");

  }

}
```

Strings are objects

A class in Java provides the code for methods, but also allows you to create what is called an *object* of that class. Among other things, an object is a structure in memory that stores values. A `String` object stores data: the value

structure in memory that stores values. A `String` object stores data: the value

of each character in the string, as well as the length of the string. Some things you can do with objects are:

1. Declare a variable to store a reference to an object of a class: `String s`.
2. Create a new object of that type:
   `s = "Rumplestiltskin";`
   Strings are somewhat special in Java; other types of objects are created using special method calls.
3. Read or modify data from the object, using dot notation to get at *instance variables*. In Java, the String method has no directly useful instance variables, so we'll see examples of this with other classes.
4. Call non-static methods on an object; these methods have access to the data within an object:
   `System.out.println(s.toUpperCase());`

```
class StringsAsObjects {
  public static void main(String[] args) {
    String s;
    s = "Rumplestiltskin";
    System.out.println(s.length());
    System.out.println(s.toUpperCase());
    System.out.println("The character at index 5 is: " + s.charAt(5));

  }

}
```

Static vs non-static methods of the String class

The Java code for the String class is stored in some file `String.java` that the Java compiler has access to, and contains code for both static and non-static method definitions.

To access static methods of the `String` class, recall that you use both the name of the class and the method name: `String.valueOf(5)`. You need the class name because there might be other methods named `valueOf()` out there.

(Indeed, there is a class called `Integer` that also has a `valueOf` method.)

Static methods do not need access to a particular string object's data. The static `valueOf` method takes a parameter that is not a String, and creates a String. Static methods are just like functions in Javascript, C, or Python. Java just uses classes to organize these functions.

Non-static methods require a string as input, and instead of passing that string as a parameter, dot-notation is used: `s.toUpperCase()`. It's possible that there might be other methods named `toUpperCase` out there. Java knows which method to call because Java knows that `s` is a String, so Java looks in `String.java` for the method.

The `char` data type

Unlike in Python or Javascript, you cannot access the characters of the string using `[]` bracket notation; in Java, bracket notation is reserved only for arrays, and you must use the `charAt` method with strings. `charAt` does not return a String: instead, it returns something called a `char`.

Data types like `int`, `double`, and `boolean` in Java are called *primitive data types*, because variables of those types are built-in, and do not have methods available to act on them. `char` is also a primitive data type, representing a single character of text.

Although the `String` data type is built-in, the `String` data type is not a primitive: it is defined by a class, with methods and an internal representation defined by that class.

Objects organize data, some of which may be composed of primitive types. A string organizes the characters in the string, which are internally represented using `char` values.

To distinguish between `String` and `char` data, Java uses single quotes for character literals and double quotes for Strings:

```
class CharExample {
  public static void main(String[] args) {
    char character = 'Z';
    String magicWord = "XYZZY";

    char anotherChar = magicWord.charAt(0);
```

```
    char anotherChar = magicWord.charAt(0);

    System.out.println(character + anotherChar);

  }
}
```

The code above prints out 178, which might surprise you. The `+` operator concatenates strings, but characters are internally represented using numbers, with a code called *Unicode*. Since the unicode value of 'X' is 88, and the unicode value of 'Z' is 90, the `+` operator adds the two up and returns an `int`.

However, you can concatenate a character to a string easily with `+`, since the `+` operator converts both operands to strings if one of the operands is a string.

Strings are immutable in Java

We will see later that variables do not actually hold objects, but *references* to objects. A reference may be loosely thought of as the address of the data storing the object in memory.

One implication is that if you pass the reference to an object to a method, the method may change the object. This is different than primitive types. If I pass the value of a variable to the `sqrt` method, I can rest assured that the `sqrt` function has no way to change that variable: the method only has the variable's value.

Java protects against strings getting changed accidentally by making objects of the String class *immutable*. You can safely pass a reference to a string to a method without worrying that the String will be changed. Notice that the `toUpperCase()` method we saw above *does not change the string*: it just creates a new string that is uppercased, and returns the result.

Therefore, although there is a way to get a character from a String using the `charAt()` method, there is no way to change a character within a string: if you want a variable to refer to some other string, you need to create a new string. There's another class in Java, `StringBuffer`, that is useful if you need to efficiently manipulate character arrays. For example, you might use a `StringBuffer` to store the text data if you were writing a text editor.

`StringBuffer` to store the text data if you were writing a text editor.

Arrays

Arrays in Java, like Python lists, Javascript arrays, and C arrays, store ordered values. Java arrays cannot be resized without creating a new array and making a copy.

WE'LL COVER THE FOLLOWING

- Declaring arrays
- Initializing arrays: shortcut notation
- Initializing arrays with `new`
- The array in `main (String[] args)`
- The `java.util.Arrays` class

Java arrays, like C arrays, are more limited than their counterparts in Python or Javascript. Java arrays cannot be resized once created, and you must declare the type of the variables that the array will store in advance. The `ArrayList` class is more flexible, and a frequent alternative to arrays, but first let's see how to use arrays.

An example of using arrays:

```
class ArrayExample {
  public static void main(String[] args) {

    // an array of ints
    int[] myNumbers = {10, 15, 20, 25, 30};

    System.out.println(myNumbers[2]);

    // arrays have an instance variable that stores
    //  the length of the array:
    System.out.println(myNumbers.length);

    // Arrays can be of any type a variable can be:
    String[] myStrings = {"Narnia", "Oz", "Neverland"};
    System.out.println("The Wizard of " + myStrings[1]);

    // arrays can be modified
    myStrings[2] = "Wonderland";
  }
```

```
}
```

Notice that although you can determine the length of a string using the method `someString.length()`, you determine the length of an array using the instance variable `someArray.length`. I consider this inconsistency to be a design flaw in Java; you'll just have to remember it.

Declaring arrays

Like any other variable, an array must be declared, and its type specified. The type for an array of `int` values is `int[]`. You can think of the empty brackets as shorthand for the word "array":

```
int[] myNumbers;
```

This is only a declaration of the variable that will hold the array. The array does not yet exist and so does not have a length.

Initializing arrays: shortcut notation

We frequently want to give an array some initial values. We have seen the shortcut method already:

```
int[] myNumbers = {10, 15, 20, 25, 30};
```

The shortcut method automatically creates an array of the neccessary length, and initializes the array with the given values. Be careful: the length of `myNumbers` in the example is now 5, and cannot be increased. Shortcut notation can only be used at the time the array is declared.

Initializing arrays with `new`

An array, like a String, is a special type of built-in object. The code

```
int[] myNumbers;
```

declares the variable `myNumbers`, but does not yet create the array object. The special keyword `new` in Java is used to create objects, including arrays.

```
class NewArray {
  public static void main(String[] args) {

    int[] myNumbers;
    myNumbers = new int[5];

    myNumbers[0] = 10;
    myNumbers[1] = 10;
    myNumbers[2] = 10;
    myNumbers[3] = 10;
    myNumbers[4] = 10;

    System.out.println("The array myNumbers has length " + myNumbers.length);

  }

}
```

The array in `main (String[] args)`

You can now see that `main` takes a single parameter, `args`, of type String array. How is it used? If you execute a program from the command line (terminal on Unix or Mac), you can pass parameters to the program. For example, the command

```
wc README.txt
```

could be typed into a unix terminal to count the words in README.txt. The first word, `wc`, is the name of the program to execute. The second word, `README.txt`, is passed as a string as the first item in the array of strings, `args`.

A similar mechanism is used in C, but since C arrays do not know their lengths, the syntax of the declaration of main is a bit different. In Python, `sys.argv` can be imported to fetch command-line arguments.

The `java.util.Arrays` class

Although arrays in Java are objects, they are built-in, and and as mentioned, are not very flexible. The `Arrays` class in the `java.util` package provides some methods that let you do things like sort, binary search, or print a string representation of an array. Unlike some other classes, like `Math` or `String` that are available by default, you must request that Java make the `Arrays` class available to you, using an *import statement* at the top of any file in which you

would like to use it:

ArraysExample

```
import java.util.Arrays;

class ArraysExample {
  public static void main(String[] args) {
    int[] myNumbers = {42, 1, 17, 27, 16};
    Arrays.sort(myNumbers);
    System.out.println(Arrays.toString(myNumbers));

  }
}
```

Notice that `Arrays.sort` works in place. Also notice that there are several overloaded `toString` methods that work on arrays containing ints, doubles, and even objects. There is no `reverse` method; you'll have to write your own or use an `ArrayList` object to store your list of data.

Conditional and logical operators

Learn to use boolean values in Java, a necessary tool for loops and conditionals.

WE'LL COVER THE FOLLOWING

- Conditional operators: >, <, >=, <=, ==, !=
- Logical operators: `&&`, `||`, and `!`
- Exercise: even in a range

A `boolean` variable can hold either the value `true` or the value `false`. Boolean values are quite useful when directing a program to take repeated actions, or to take actions only under some circumstances.

Note that Java's `true` and `false` values are written with lowercase, unlike Python's.

```
class BooleanValues {
  public static void main(String[] args) {
    boolean b = true;
    System.out.println(b);
    System.out.println(false);
  }
}
```

Conditional operators: >, <, >=, <=, ==, !=

Conditional operators work just like they do in most common programming languages. You can compare ints, doubles, and Strings, yielding a `true` or `false` value.

A very typical coding error is to type `=` when you meant `==`: assigning a value to a variable when you meant to test the value of that variable. Be careful –

the Java compiler will not catch this error for you:

```
class AssignmentError {
  public static void main(String args[]) {
    int x = 5;
    System.out.println(x = 4);
  }
}
```

Logical operators: `&&`, `||`, and `!`

The *and* operator is written as `&&` in Java. It yields `true` if both operands are `true`. The *or* operator `||` works as you would expect from other languages, too. The *not* operator, `!` precedes a boolean `true` or `false` value that you would like to negate.

Exercise: even in a range

To practice writing boolean expressions that test if a variable satisfies several conditions, complete the function `evenInRange` below so that the function returns true if the parameter `x` is an even number in the range 24...32 (inclusive of the values 24 and 32). You should only need to write a single line of code.

EvenInRange.java | Sample solution

```
class EvenInRange {

  public static boolean evenInRange(int x) {
     // you write this part (a one-line return statement)
  }

  public static void main(String[] args) {
    System.out.println(evenInRange(0));
    System.out.println(evenInRange(24));
    System.out.println(evenInRange(25));
    System.out.println(evenInRange(34));
  }
}
```

if-statements

Learn the syntax for if-statements in Java.

WE'LL COVER THE FOLLOWING

- Exercise: thermometer
- Exercise: I see a wall!
- Exercise: Coin flip
- `else` and `else if` statements

The keyword `if` can be used to run a block of code only if some condition has the boolean value `true`. The syntax is like that of C or Javascript: the conditional expression must be wrapped in parentheses, and the statements to be executed should be in curly braces. If you are familiar with those languages, you may wish to skip this section.

Here are a few examples:

```
class IfExamples {
  public static void main(String args[]) {
    if(true) {
      System.out.println("This code gets executed.");
    }
    if(false) {
      System.out.println("This code does not.");
    }

    if(5 > 3) {
      System.out.println("This code gets executed, too.");
    }

    if(3 > 5) {
      System.out.println("This code does not.");
    }
  }
}
```

Exercise: thermometer

Write an if-statement in the following code that checks if the variable `temperature` has a negative value, and if so, prints out, “It’s very, very cold!”

Then change the value of temperature to ensure that nothing is printed out.

Finally, add a second if-statement to check if temperature is non-negative, and if so, prints out “It’s not so cold.”

Thermometer.java | Sample solution

```
class Temperature {

  public static void main(String[] args) {

    int temperature = -5;
  }
}
```

Exercise: I see a wall!

A method call causes some code to be executed, and that code may return a result for use. For example, the method call `Math.sqrt(9.0)` computes and makes available the value `3.0` for printing, storing, or some other use.

The type of value that the method returns depends on the particular method. `Math.sqrt(9.0)` returns a number. In the code below, you can use the method `r.blocked()`, which returns a boolean value of `true` or `false`, depending on whether or not there is a wall in front of the robot `r`.

In the following code, write an if-statement that prints out “I see a wall!” after the robot drives a few steps, if there is a wall in front of the robot. Try changing the movement commands to verify that the robot doesn’t report a wall if there is not one.

SeeWall.java | Sample solution

```
class RobotDemo {

        public static void main(String[] args) {

    SimpleRobot r = new SimpleRobot();

    r.forward();
    r.right();
    r.forward();
    r.left();

    // Add your if-statement here:

  }
}
```

Exercise: Coin flip

The code below picks a random number that is either 0 or 1, and stores the result in the variable `n`. Add some if statements so that the program randomly prints out “Heads” or “Tails”. Run the program several times to make sure that you eventually get both outcomes.

CoinFlip.java | Sample solution

```
import java.util.Random;

class CoinFlip {

        public static void main(String[] args) {
    // create a new random number generator object
    Random rand = new Random();

    // pick a random number between 0 and 1:
    int n = rand.nextInt(2);
    System.out.println("Picked random value " + n);

  }
}
```

`else` and `else if` statements

Java also has `else` and `else if` statements; they work as you expect from other languages. (Python uses `elif`, but like C and Javascript, Java uses the syntax `else if`.)

In the coin flip exercise, replace the second if statement with an `else` statement, and in the thermometer exercise, write an if-ladder that uses `else if` statements.

while-loops

Learn the syntax of while-loops in Java.

WE'LL COVER THE FOLLOWING

- Exercise: charge!
- Translation exercise: factor

while-loops in Java work exactly the same as in Python, Javascript, and C, although the syntax is different than that of Python.

The examples and exercises are worth working through anyway, to gain comfort with code organization in Java. Here is a simple example of using a while-loop to count in Java:

```
class WhileExample {
  public static void main(String[] args) {
    int number = 0;
    while(number < 100) {
      System.out.println(number);
      number += 2;
    }
  }
}
```

Exercise: charge!

In Java, while-loops are rarely used for counting, since for-loops provide a briefer syntax. But while-loops are great for doing an action some unspecified number of times until a condition changes. In the code below, write a static method `charge` that causes the robot to drive forwards until it has gone either `maxDist` squares or until it hits a wall, whichever is sooner.

You may call the method `blocked()` on the robot; `blocked()` returns `true` if there is a wall in front of the robot, and false otherwise.

ChargingRobotDemo.java | Sample solution

```
class ChargingRobotDemo {

  public static void charge(SimpleRobot r, int maxDist) {
    // you write this part
  }

  public static void main(String args[]) {
    SimpleRobot r = new SimpleRobot();
    r.forward();
    r.right();
    charge(r, 3);
  }

}
```

You probably noticed that the syntax for calling `charge()` on a robot is different than the syntax for calling `forward()`, since the robot is passed as the first parameter to `charge`, rather than appearing before the dot. This is because `charge` is a static method of the current class, while `forward` is a method of the `SimpleRobot` class. We'll see how to write non-static methods soon.

Translation exercise: factor

It's good practice to translate code from one language to another. Here is some Python code that computes the integer factors of a number. Write Java code to compute the factors in the second tab; while you are at it, put the code inside a nice `public static void` method `printFactors` that takes an integer parameter and prints the integer factors on the screen. You should be fine even if you do not know Python.

factor.py | PrintFactors.java | Sample solution

```
number = 42

possible_factor = 1
while possible_factor <= number:
    if number % possible_factor == 0:
        print( str(possible_factor) + " is a factor of " + str(number) + "." )
```

```
    possible_factor = possible_factor + 1

print( "And that's all the factors of " + str(number) + "." )
```

It would be even nicer to have a method that returns a list of factors, rather than printing the numbers out on the screen. One way to return the list of numbers would be to create an array; unfortunately, since we don't know how many factors there will be ahead of time, we'd either have to create an array that was too large, or first count the factors. Later, we'll see how to use an `ArrayList` object that will serve as an extensible list.

for-loops

Learn how to use for-loops for counting and indexing.

WE'LL COVER THE FOLLOWING

- Exercise: blast off

For-loops in Java are identical to those in C and Javascript, and if you are familiar with those languages, you may skip this section. If you are most familiar with Python, you'll want to work through this lesson carefully. Here's a for-loop in Java:

ForExample.java

```
class ForExample {
  public static void main(String[] args) {
    for(int i = 1; i < 11; i++) {
      System.out.println(i);
    }
  }
}
```

Python for-loops iterate over elements of a list. The for loop shown above does not. Let's first look at a while-loop that has a very similar intention as the above for-loop.

```
class WhileExample {
  public static void main(String[] args) {
    int i = 1;            // initialize a variable
    while(i < 11) {       // test a condition
      System.out.println(i);
      i++;                // modify the variable value
    }
  }
}
```

The while-loop shows an obvious way to count by incrementing the variable `i`, and the pattern is very standard. Create and initialize a variable `i`, test if `i` satisfies some condition, execute the body, increment `i`, and so forth. Here's the basic structure in pseudocode:

```
init
while(condition) {
  // body
  modify
}
```

The for-loop looks like this:

```
for(init; condition; modify) {
  // body
}
```

The for-loop works like this:

1. Execute the instruction **init**
2. Test the **condition**
3. Execute the body
4. Execute the instruction **modify**
5. Go back to step 2.

Exercise: blast off

Replace the while-loop with a for-loop:

BlastOff.java | Sample solution

```
class BlastOff {
  public static void main(String[] args) {
    int i = 10;
    while(i > 0) {
      System.out.println(i);
      i--;
    }
    System.out.println("Blast off!");
  }
```

```
}
```

There's a slight difference between the while-loop and the for-loop in the sample solution. The variable `i` created in the initialization section of the for-loop is in scope only for the for-loop body, and goes out of scope after the body. On the other hand, the while-loop creates the variable `i` before the loop, so `i` is available after the loop as well.

foreach-loops

Learn how to use foreach-loops to loop over list-like structures.

Python for-loops loop over sequences; a list is one type of sequence. This can be very convenient, and Java 8 introduced a similar capability, called a foreach-loop. In spite of the name, a foreach-loop uses the keyword `for`. Here is an example:

```
class ForEachExample {
  public static void main(String[] args) {
    int[] primes = {2, 3, 5, 7, 11, 13};
    for(int p: primes) {
      System.out.println(p);
    }
  }
}
```

The instruction before the colon creates a variable. That variable then takes on every value in the **collection** after the colon. An array is a particular type of collection; so are ArrayLists and other data structures that we will see later.

Only the value of an array item is copied into `p` above; changing `p` does not change the array.

Java 8 also introduced a method `.forEach()` of collections. It is quite nice, but since we have not discussed collections in detail, we will leave `.forEach()` for another day.

Classes define objects

A class is used to define the custom data structure that makes up an object of that class.

WE'LL COVER THE FOLLOWING

- A class declares what instance variables an object of that class will contain
- Creating an object of a class
- Constructors
- The special variable `this`
- Exercise: Ball class

We have seen that classes are the organizational unit of a Java program. Source code for methods is always within a class, and we might expect related methods to be in the same class.

However, classes do much more than this, and are the heart of object-oriented programming in Java:

1. A class is used to define the **custom data structure** that makes up an object.
2. Classes define **methods** that act on those objects.

Here's an example of creating a custom data type, `Circle`, to store information about a circle:

```
// include educative's simple graphics library:
import com.educative.graphics.*;

class Circle {
  public int x;
  public int y;
  public int r;
}

class CircleTest {
  public static void main(String[] args) {
```

```
    Circle circ;   // declare a variable to store a reference to Circle object

    // create a new Circle object in memory, and store a reference to it:

    circ = new Circle();

    // set values for the instance variables for the circle:
    circ.x = 100;
    circ.y = 100;
    circ.r = 50;

    // Create a Canvas object for drawing on the screen:
    Canvas c = new Canvas(200, 200);
    c.fill("yellow");
    c.stroke("black");

    // use instance variables of the the circle object when calling
    //  the circle method of the canvas (used to draw the circle)
    c.circle(circ.x, circ.y, circ.r);
  }

}
```

Notice that there are two different classes defined in this file. Typically in Java, each class would get its own file, with the name of the file `ClassName.java`. So `Circle` might be defined in a class `Circle.java`. This is a good convention and we should follow it; the classes are in the same file here only because it is easier to read on the web page.

A class declares what instance variables an object of that class will contain

Just like in Javascript, Python, or C++, an object is a data structure stored on the heap area of memory, containing data elements. These elements are called **instance variables** in Java. A class definition lists out the types and names that each object of that class will have. So each `Circle` object will contain three `int` values: `x`, `y`, and `r.`.

In Python or Javascript, instance variables of an object are created using a special constructor method. Java also has constructors, but in Java, variables must be declared before they can be used or assigned values to – even the constructor cannot create a new variable that has not been explicitly declared.

In Java, **every instance variable must be declared in the class.** Notice that the declarations for the instance variables occur **outside** of any method in `class Circle`: the class just lists them out. That's because variables declared inside of method definitions are local variables, and are destroyed once the method returns.

Also notice the keyword **public** before each of these instance variables. This indicates that methods of other classes can access these variables. We'll see more later about the difference between public and private variables or methods.

Creating an object of a class

Although a class describes the instance variables of an object, no instance variables exist until an object of the class is created. The command `new`, which must be followed by a call to the constructor of a class, creates space on the heap for the object, and returns a reference to that object:

```
Circle circ;
circ = new Circle();
```

You can think of the reference as the address of the object in memory. So the above code creates a new object on the heap with the code `new Circle()`, and stores a referene to that object in the variable `circ`. Like every other variable in Java, `circ` must have a type. The declaration `Circle circ` creates the variable and indicates that it will store a reference to an object of type `Circle`.

You might notice that there is no definition for a function or method `Circle()` anywhere in the code. Java creates a *default constructor* function for you automatically, with the same name as the name of the class.

Constructors

Object-oriented design drives the structure of the Java language. Classes and objects are intended to model specialized types of data; in this case, a circle. When you first create an object using the default constructor, space in

memory is allocated for the variables, but they are not assigned any values.

You almost always want to write your own constructor, which will let you give initial values to instance variables. Here's an example:

```
import com.educative.graphics.*;

class Circle {
  public int x;
  public int y;
  public int r;

  public Circle(int initX, int initY, int initR) {
    this.x = initX;
    this.y = initY;
    this.r = initR;
  }
}

class CircleTest {
  public static void main(String[] args) {
    Circle circ;   // declare a variable to store a reference to Circle object

    // create a new Circle object in memory, and store a reference to it:
    circ = new Circle(100, 100, 50);

    // Create a Canvas object for drawing on the screen:
    Canvas c = new Canvas(200, 200);
    c.fill("yellow");
    c.stroke("black");
    c.circle(circ.x, circ.y, circ.r);
  }
}
```

Some rules to remember:

1. The constructor has the same name as the class.
2. The constructor should initialize all instance variables of the object, possibly using some passed parameters.
3. The constructor has access to a reference to the newly created object, stored in the special variable `this`.

Like in other languages, constructors are a nice place to check for good values. For example, you might verify that the value of the radius for the circle is non-negative, and if not, halt the program.

The special variable `this`

Instance variables of an object are accessed using dot-notation, which requires a reference to an object, a dot, and the name of the instance variable.

The job of the constructor is to initialize the instance variables of some object. That object was just created with the command `new`, followed by a call to the constructor. In order to access the instance variables, the constructor requires a reference to that newly created object. The variable `this` is maintained by `Java`, and contains the needed reference.

The code `this.x = initX` therefore takes the value of the constructor parameter `initX`, 100 in this case, and stores it into the instance variable `x` of the object referenced by `this`.

The variable `this` is essentially identical to the variable `self` in Python, and somewhat related to the variable `this` in Javascript: each of these variables stores a reference to some object, which can be used to manipulate the instance variables of that object.

In Python, the constructor is defined by writing the body of the special method `__init__(self, ...)`, and the first parameter, `self`, is written into the parameter list. Java constructors are named using the name of the class, and the parameter `this` is not included in the parameter list, but is nonetheless available automatically.

The variable `this` is also available in non-static method calls other than the constructor, as we will see shortly, even though `this` is not an explicit parameter to those methods.

Sometimes you will see Java code that omits the `this.` when accessing instance variables. For now, I recommend that you always use `this` when accessing instance variables in the constructor: the keyword makes it extremely clear that you are accessing an instance variable, and not some local variable or parameter to the call.

Since you will see it code written by others, here's an example of omitting `this`:

```
class Circle {
```

```
  public int x;
  public int y;
  public int r;


  public Circle(int initX, int initY, int initR) {
    x = initX;
    y = initY;
    r = initR;
  }
}
```

I chose to name the first parameter to the constructor `initX`. Some might choose to call that parameter `x`, since it will contain the value of the `x` coordinate of the circle's center. There's nothing wrong with that choice, but if parameters and instance variables have the same name, you cannot omit the word `this.` from the assignment to the instance variable:

```
  public Circle(int x, int y, int r) {

    // Notation with "this." on the left-hand side makes it clear
    //  that we are assigning the values of parameters into instance variables.
    this.x = x;
    this.y = y;
    this.r = r;
  }
```

Exercise: Ball class

Write a class `Ball` that will be used to model a ball bouncing on the screen. Your ball should have five instance variables: a `String` describing the color of the ball, and `int` values describing the x location, a y location, and x and y components of the velocity of the ball, vx and vy. (For now, the velocities will just be set to zero. We will use them in the next lesson.)

All of the instance variables should be declared as public, so that they may be accessed by the `drawBall` method in the main `BallExample` class.

You'll need to:

1. declare the appropriate variables;
2. write a constructor.

Read the code in `main` to see how your constructor should be organized.

BallExample.java | Sample solution

```
import com.educative.graphics.*;

class Ball {
  // you write this part:
}

class BallExample {
  public static void drawBall(Canvas canvas, Ball ball) {
    canvas.fill(ball.color);
    canvas.stroke("black");
    canvas.circle(ball.x, ball.y, 10);
  }

  public static void main( String args[] ) {
    Canvas c = new Canvas(200, 200);

    // create a red ball at location (20, 30) with 0
    //  x and y velocity:
    Ball b = new Ball("red", 20, 30, 0, 0);

    drawBall(c, b);
  }
}
```

Classes contain methods

Classes contain the code for methods that act on objects; methods have access to the object using the special variable "this".

WE'LL COVER THE FOLLOWING

- static vs. non-static methods
- Exercise: add a `move` method
- Exercise: add a bounce method

Here is some code to create a custom `Ball` class that is used to store information about a ball that will bounce on the screen. The `main` method of `BallExample` creates a `Ball` object using the `new` keyword and a call to the constructor. Read the code carefully now:

```
import com.educative.graphics.*;

class Ball {
  public String color;
  public int x;
  public int y;
  public int vx;
  public int vy;

  public Ball(String color, int x, int y, int vx, int vy) {
    this.color = color;
    this.x = x;
    this.y = y;
    this.vx = vx;
    this.vy = vy;
  }
}

class BallExample {
  public static void drawBall(Canvas canvas, Ball ball) {
    canvas.fill(ball.color);
    canvas.stroke("black");
    canvas.circle(ball.x, ball.y, 10);
  }

  public static void main( String args[] ) {
    Canvas c = new Canvas(200, 200);
```

```
    // create a red ball at location (20, 30) with 0
    //  x and y velocity:

    Ball b = new Ball("red", 20, 30, 0, 0);

    drawBall(c, b);
  }
}
```

The `BallExample` class also has a method `drawBall` that accesses instance variables of the `Ball` object and uses them, together with a `Canvas` object to draw a circle in the right place on the screen.

Classes group together methods. It seems like the code for `drawBall` might naturally be in the `Ball` class. Like this:

```
import com.educative.graphics.*;

class Ball {
  public String color;
  public int x;
  public int y;
  public int vx;
  public int vy;

  public Ball(String color, int x, int y, int vx, int vy) {
    this.color = color;
    this.x = x;
    this.y = y;
    this.vx = vx;
    this.vy = vy;
  }

  public void draw(Canvas canvas) {
    canvas.fill(this.color);
    canvas.stroke("black");
    canvas.circle(this.x, this.y, 10);

  }
}

class BallExample {
  public static void main( String args[] ) {
    Canvas c = new Canvas(200, 200);

    Ball b = new Ball("red", 20, 30, 0, 0);

    // call the draw method of the ball class
    b.draw(c);
  }
}
```

The `BallExample` class is now much simpler, since we've moved some code into `Ball`. There are a few things to notice:

1. Just like in Python or Javascript, the method is called with a reference to an object, followed by a dot, followed by the method call: `b.draw(c)`.

2. In the code for `draw`, the special instance variable `this` is available, and contains a reference to the object that appeared before the dot. So `this` refers to the same object that `b` refers to.

3. In Python, the first parameter in the method definition is the special variable `self`. In Java, the variable `this` plays the same role, but is omitted from the parameter list.

static vs. non-static methods

The `draw` method is a **non-static** method. Non-static methods are the default in Java, and have access to the keyword `this`, providing access to a particular object: the one referenced before the dot. We can tell that `draw` is non-static because there is no keyword `static` before the return type `void`.

A class may included both **static** and ordinary, or non-static, methods. Related static methods are grouped together by a class. For example, the `Math` class contains the `sqrt` method to take square roots, and the `pow` method to raise a number to a power. Neither method requires an object, but every method must occur within a class, and the `Math` class is a nice place to put both of these methods.

You can think of Java static methods as being like ordinary functions in Python or Javascript. Java non-static methods are like methods in Python or Javascript.

Exercise: add a `move` method

Use the `Ball` class to create an animation of a ball moving across the screen. To do this, write a method `public void move` that takes no parameters. The method should add the current x velocity `vx` to the current x location `x`, and add the current y velocity to the current y location. The provided code calls

`move` in a loop to move the ball across the screen for 50 time steps.

BallExample.java | Sample solution

```
import com.educative.graphics.*;

class Ball {
  public String color;
  public int x;
  public int y;
  public int vx;
  public int vy;

  public Ball(String color, int x, int y, int vx, int vy) {
    this.color = color;
    this.x = x;
    this.y = y;
    this.vx = vx;
    this.vy = vy;
  }

  public void move() {
    // you write this part:
  }

  public void draw(Canvas canvas) {
    canvas.fill(this.color);
    canvas.stroke("black");
    canvas.circle(this.x, this.y, 10);

  }
}

class BallExample {
  public static void main( String args[] ) {
    Canvas c = new Canvas(200, 200);

    Ball b = new Ball("red", 20, 30, 1, 2);

    for(int i = 0; i < 50; i++) {
      // clear the screen and draw the ball
      c.clear();
      b.draw(c);

      // use the ball velocity to update x and y position
      b.move();

      // instruct the drawing code to pause for 20 milliseconds
      c.wait(20);

    }
  }
}
```

Exercise: add a bounce method

Write a method `bounce` that causes the ball to switch velocity directions if the ball is about to move off of the 200 x 200 pixel canvas. If the x coordinate is too large or too small, multiply `vx` by -1. If the y coordinate is too large or too small, multiply `vy` by -1.

BouncingBall.java | Sample solution

```
import com.educative.graphics.*;

class Ball {
  public String color;
  public int x;
  public int y;
  public int vx;
  public int vy;

  public Ball(String color, int x, int y, int vx, int vy) {
    this.color = color;
    this.x = x;
    this.y = y;
    this.vx = vx;
    this.vy = vy;
  }

  public void move() {
    this.x += this.vx;
    this.y += this.vy;
  }

  // add your bounce method here:

  public void draw(Canvas canvas) {
    canvas.fill(this.color);
    canvas.stroke("black");
    canvas.circle(this.x, this.y, 10);

  }
}

class BouncingBall {
  public static void main( String args[] ) {
    Canvas c = new Canvas(200, 200);

    Ball b = new Ball("red", 60, 30, 1, 2);

    for(int i = 0; i < 500; i++) {
      // clear the screen and draw the ball
      c.clear();
      c.fill("lightblue");
      c.rect(0, 0, 199, 199);
      b.draw(c);

      b.move();
      b.bounce();
```

```
      // instruct the drawing code to pause for 20 milliseconds
      c.wait(20);


    }
  }
}
```

Access specifiers: public and private data and methods

Learn how to use the keywords public and private, and how to use getter and setter methods to maintain consistent object state.

WE'LL COVER THE FOLLOWING

- Setter methods
- Exercise: `setLocation` method
- The keyword `private`
- Getter methods
- Exercise: methods `getX`, `getY`, `getRadius`
- Most instance variables should be private
- Some methods are public, and others are private
- Think hard about which methods you should make public

Instance variables of an object can be accessed using dot notation, just like in Python, Javascript, or C++. Here's the simple example of defining a `Circle` class, which allows `Circle` objects to be created. Each circle object has the instance variables `x`, `y`, and `r`:

```
// include educative's simple graphics library:
import com.educative.graphics.*;

class Circle {
  public int x;
  public int y;
  public int r;
}

class CircleTest {
  public static void main(String[] args) {
    Circle circ;
    circ = new Circle();

    // set values for the instance variables for the circle:
    circ.x = 100;
    circ.y = 100;
```

```
    circ.r = 50;

    Canvas c = new Canvas(200, 200);

    c.fill("yellow");
    c.stroke("black");

    c.circle(circ.x, circ.y, circ.r);
  }
}
```

We can access the radius of the circle with `circ.r` in Java, just like in Python or Javascript, and we've done so in the above code.

However, in Java, it is considered **bad form to directly access instance variables** from methods outside the class. Why?

1. Internal representations change. If I decide later that the radius of the circle should be a double, and that the variable name should be `radius`, I might remove the instance variable `r`, breaking every piece of code that depends on it.

2. No error checking is performed when modifying an instance variable directly. For example, if I write the code `circ.r = -50`, then the circle is in an inconsistent state.

Setter methods

If we are not allowed to set the value of `r` directly from outside the `Circle` class, how should we change the radius of the circle? One way of modifying instance variables is a **setter** method. Such a method may be called from outside the class, and handles the management of instance variables as needed. For example:

```
// include educative's simple graphics library:
import com.educative.graphics.*;

class Circle {
  public int x;
  public int y;
  public int r;

  public void setRadius(int radius) {
    if(radius < 0) {
      throw new IllegalArgumentException("Circle radius must be non-negative.");
```

```
    }
    this.r = radius;
  }

}

class CircleTest {
  public static void main(String[] args) {
    Circle circ;
    circ = new Circle();

    // set values for the instance variables for the circle:
    circ.x = 100;
    circ.y = 100;
    circ.setRadius(20);

    Canvas c = new Canvas(200, 200);
    c.fill("yellow");
    c.stroke("black");

    c.circle(circ.x, circ.y, circ.r);
  }
}
```

Run the above code. Then change the argument in `circ.setRadius(20)` to -20, and run the code again. You'll see that an exception is generated and the code crashes. Good! Now the programmer knows that there is a bug and can fix it.

To generate an error message, the method `setRadius` makes use of a Java command called `throw`. The keyword `throw` generates an error which, if not handled, causes the program to crash. We will not go into details of how `throw` (and its sister `catch`) work, but once you are more comfortable with the basics of Java, exception handling is a useful topic to explore.

Exercise: `setLocation` method

In the code below, write a method `setLocation` that takes two parameters, an x and y coordinate, and sets the instance variables `x` and `y` of the circle to those coordinates. You do not need to do any error checking, since coordinates may legally have negative values. Also write a constructor that lets the initial values be set when the circle is created.

CircleExample.java | Sample solution

```
// include educative's simple graphics library:
```

```
import com.educative.graphics.*;

class Circle {

  public int x;
  public int y;
  public int r;


  // Write your constructor here:

  // Write your setLocation method here:

  public void setRadius(int radius) {
    if(radius < 0) {
      throw new IllegalArgumentException("Circle radius must be non-negative.");
    }
    this.r = radius;
  }

}

class CircleExample {
  public static void main(String[] args) {

    Canvas c = new Canvas(200, 200);
    c.fill("yellow");
    c.stroke("black");

    Circle circ;
    circ = new Circle(100, 50, 40);
    c.circle(circ.x, circ.y, circ.r);


    circ.setLocation(100, 125);
    circ.setRadius(20);

    c.circle(circ.x, circ.y, circ.r);


  }
}
```

In the sample solution, I called the methods `setLocation` and `setRadius` from the constructor. Although this is not required, it's nice to make use of the error checking in `setRadius` when setting the initial radius using the constructor, and if I later add new behavior to `setLocation`, then the constructor will make use of it automatically.

The keyword `private`

The *principle of least privilege* suggests that we should give code the absolute

minimum capabilites to do the needed job. Direct access to variables is a

powerful capability, and often not needed. I do not need the capability of changing the radius of a circle to be a negative number.

Now that we have methods to set the location and radius of the circle, we don't ever need to write lines of code like `circ.r = 20`. In fact, we'd like to force programmers to use `setRadius` rather than accessing the instance variable directly, so that we can make sure that error checking is always performed.

So far, we have always made instance variables `public`. This means that any code can access those variables with a reference to the object and dot notation. `public` is called an *access specifier*.If we change the *access specifier* to `private`, then only code within the `Circle` class can access the instance variable. `circ.r = 20` is an illegal instruction if `r` is a private instance variable.

Here's the code with the access specifiers changed to private:

```
// include educative's simple graphics library:
import com.educative.graphics.*;

class Circle {
  private int x;
  private int y;
  private int r;

  public Circle(int x, int y, int r) {
    this.setLocation(x, y);
    this.setRadius(r);
  }

  public void setLocation(int x, int y) {
    this.x = x;
    this.y = y;
  }

  public void setRadius(int radius) {
    if(radius < 0) {
      throw new IllegalArgumentException("Circle radius must be non-negative.");
    }
    this.r = radius;
  }

}

class CircleExample {
  public static void main(String[] args) {
```

```
    Canvas c = new Canvas(200, 200);
    c.fill("yellow");
    c.stroke("black");


    Circle circ;
    circ = new Circle(100, 50, 40);
    c.circle(circ.x, circ.y, circ.r);

    circ.setLocation(100, 125);
    circ.setRadius(20);

    c.circle(circ.x, circ.y, circ.r);

  }
}
```

Run the above code. Notice that the compiler gives you some error messages. Specifically, there is something wrong with the line of code `c.circle(circ.x, circ.y, circ.r);`.

Aha! We used the `private` access specifier: we cannot modify or even use the value of the instance variables `x`, `y`, or `r` from outside the class.

One way to handle this problem would be to write a method `public void draw` within the `Circle` class, like we did for the `Ball` class in the previous lesson. The `draw` method is inside the `Ball` class and has access to the instance variables of the object referred to by `this`. In general, the `this` keyword is the only way to access a private instance variable. (You may eventually omit the keyword `this`, leaving it implied, but for now, you should use it.)

Try writing a draw method and using it to replace the code `c.circle(circ.x, circ.y, circ.r);`. Make sure your draw method accepts a reference to a Canvas object, since it will need it to call `circle`.

Getter methods

What if you really want to know the radius of some Circle object from outside of the `Circle` class? If the instance variables are `private`, and they should be, `circ.r` is unavailable. One way to handle this is to write a method `getRadius` that returns the value of the radius. Such a method is called a **getter**. Let's do it:

Exercise: methods `getX`, `getY`, `getRadius`

Write getter methods that get the values of `x`, `y`, and `r` instance variables. Notice that each of these methods returns an `int`, so make sure to specify that in the method header. The body of each method should be only one line long.

CircleExample.java | Sample solution

```
// include educative's simple graphics library:
import com.educative.graphics.*;

class Circle {
  private int x;
  private int y;
  private int r;

  public Circle(int x, int y, int r) {
    this.setLocation(x, y);
    this.setRadius(r);
  }

  public void setLocation(int x, int y) {
    this.x = x;
    this.y = y;
  }

  public void setRadius(int radius) {
    if(radius < 0) {
      throw new IllegalArgumentException("Circle radius must be non-negative.");
    }
    this.r = radius;
  }

  // write your getter methods here:

}

class CircleExample {
  public static void main(String[] args) {

    Canvas c = new Canvas(200, 200);
    c.fill("yellow");
    c.stroke("black");

    Circle circ;
    circ = new Circle(100, 50, 40);

    System.out.println("The radius of circ is " + circ.getRadius());

  }
}
```

Writing getter methods may seem annoying, but you do not need to write a getter method for every instance variable: just for the ones you want direct

access to. And in fact, getter methods have an additional features: they can separate internal representation of data from the external *view* of that data.

For example, perhaps you have written a thermometer class. Internally, you might represent the temperature using the Kelvin scale. You could write getter methods `getCelsius` and `getFahrenheit` to do the conversions and access the data.

Most instance variables should be private

Almost all instance variables should be `private`. This ensures that access to instance variables is done in a controlled fashion, and creates nice barriers between different modules of code. If an instance variable is `private`, you can debug the class you are working on secure in the knowldege that the instance variable's value was set somewhere within the current class.

The only time you might have a public instance variable is for a very small class that packages simple values together. For example, the class `java.awt.geom.Point2D` provided by Java has public `x` and `y` instance variables. These variables represent the integer coordinates of a point and are not restricted in any way.

Some methods are public, and others are private

So far, we have only seen public methods. Getter and setter methods must be public, of course, and so must constructors.

Sometimes it is useful to factor a method into smaller methods, however, and those smaller methods may not have usefulness beyond the current small piece of code you are working on. In this case, these methods may be declared `private`.

Think hard about which methods you should make public

In general, you should aim to have only a few public methods for each class: these methods make up the public, documented interface for using the class.

Once you have created a public method, other people may use that method, and you must always make sure that as your code evolves, that method is still

and you must always make sure that as your code evolves, that method is still

available. **If you change the name, parameters, or behavior of a public method, you will break all code that makes use of that method.**

Tetrominos: the game

Get ready to write a falling-blocks puzzle game.

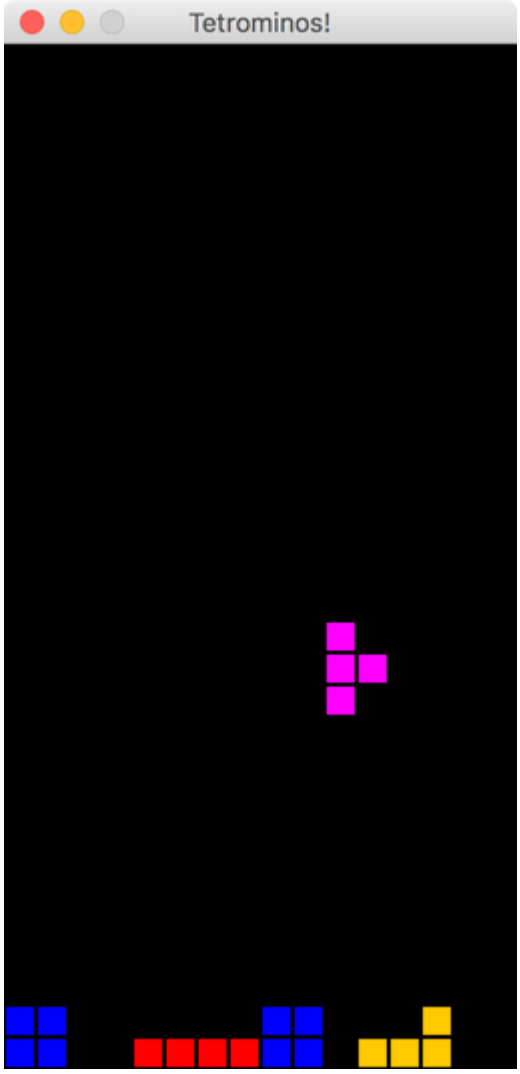
WE'LL COVER THE FOLLOWING

- Graphics and user interface libraries
- Code design
  - Modeling blocks with the `Block` class
  - Modeling the board with a `Board` class
  - Modeling the active tetromino with the `Piece` class
  - Next steps

You now have the tools and techniques needed to write interesting and complete Java programs. In the lessons of this tutorial, we'll walk through writing a complete falling-blocks puzzle game. To do this, you'll need to install a Java development environment on your computer, but first, let's talk about the rules of the game and some elements of how we might design the code.

The basic rules of the game are the same as for the game Tetris™; you can easily find several browser-based implementations of Tetris with a web search.

A *tetromino* is a shape composed of four blocks. Here's a screenshot of the game in action, with five tetrominos on the screen; there are only a few types of tetrominos, and they are colored by type in the screenshot.



Tetrominos fall downwards, and can be rotated or moved side to side under keyboard control. I used the `s` key to move a tetromino to the left, the `f` key to move the tetromino to the right, the `d` key to rotate to the left, and the `g` key to rotate to the right.

If a horizontal row is completely filled with blocks, that row is destroyed and all blocks above that row fall downwards one row. The goal of the game is to destroy as many rows as possible before the board becomes so cluttered that a new piece cannot start falling from the top.

Graphics and user interface libraries

There are a few things you'll need to know to get started. First, you'll need to know about how to draw things on the screen. So far, you've used the `io.educative.graphics` package, but this package is only for web demonstrations. You'll also need to know how to collect keyboard input from the user in order to control the motion of the pieces.

I used the `Swing` and `awt` libraries included with Java to do the drawing, although there are other choices. These are very large libaries; we'll look at the pieces we need in a few lessons.

Code design

Before writing a program of any interesting size, you should sit down with pencil and paper and plan out the code design. With Java, this usually entails mapping out what classes you'll need and how they will work together. You should also think about what instance variables and methods each class will include.

Don't implement anything yet! Work through the design carefully, and read through the next remarks. The clearer a picture you have before you start coding, the better it will go. I usually take a day or two to think over my plans, and you should too.

Modeling blocks with the `Block` class

A good approach to design works both from the bottom up and from the top down, seeking to meet in the middle. For example, we can see in the game that there are square blocks. Those blocks are sometimes arranged into a falling tetromino, and sometimes they are just left over on the board from a previous tetromino. At the lowest level, we need to model those blocks somehow.

Since there are blocks of different colors, each block should keep track of what color it is. I created a `Block` class with an instance variable storing that color.

The game board will keep track of the location of each block somehow. Therefore, I decided that a block didn't need an instance variable to keep track of where it was.

Modeling the board with a `Board` class

In my design, most of the work for the game is handled by the `Board` class. The Board:

1. contains an array of Squares representing left-over squares on the board;

2. contains an active tetromino object that is currently falling;
3. has a method to draw the board;
4. has a method to move the active piece downwards each game turn;
5. has methods to slide the active tetromino piece left or right or rotate it;
6. has a method for testing for collisions between squares as pieces are moved;
7. has a method for checking for complete rows, and for destroying them as needed.

That's a lot, and it's too much to implement all at once. For an implementation, just putting some blocks into an array and drawing them on the screen would be a great first step.

Modeling the active tetromino with the `Piece` class

There are several types of tetromino. I used the `Piece` class to construct and store various types of tetrominos.

As usual, it's good to work out a complete design for the Piece class, but be flexible. When you implement a class to represent the Piece, start with something simple. Perhaps you might start with a falling piece that is just a single block, rather than four blocks. Then add a method to translate the piece, maybe add the color, and a `draw` method.

Next steps

Ok, with design thought through, and a tentative plan for what you might implement first, it's time to start coding. The next lessons will:

1. Introduce you to an IDE for developing code.
2. Introduce you to the AWT and Swing and graphics and user interface libraries.
3. Provide an implementation roadmap.

Getting started with the JDK and an IDE

Install and use the tools you'll need to start writing complete Java programs on your own machine.

WE'LL COVER THE FOLLOWING

- Java
- Eclipse IDE
  - Hello World in Eclipse
- Exercise: creating a new class

To write Java code on your own computer, rather than in the web browswer, you'll need a Java Development Kit (JDK) to compile and run Java code, and some way of editing Java files.

Java

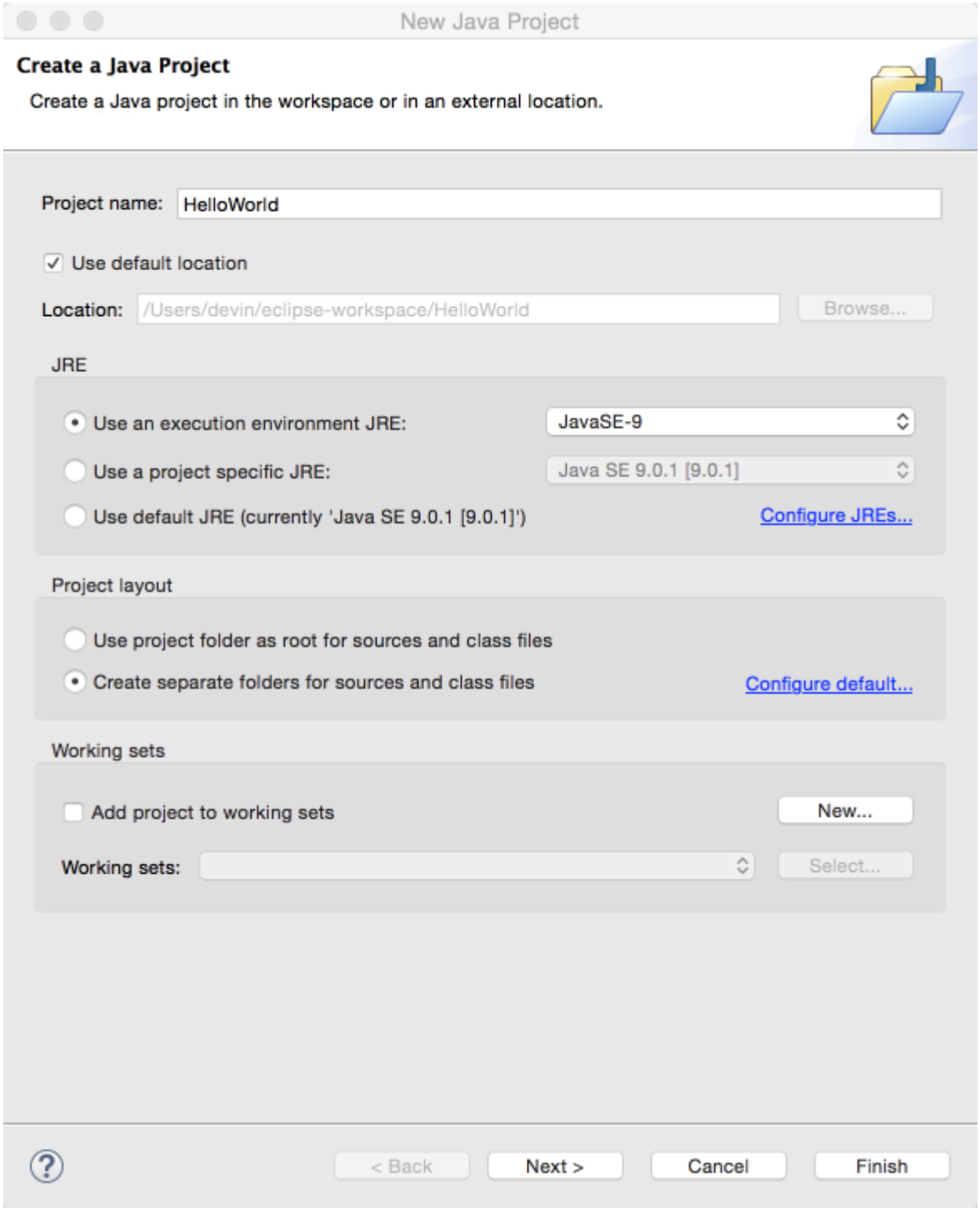
You'll need to download and install the JDK first; the current version is Java 9. You can download it from the download link on Oracle's Java page.

Eclipse IDE

Although a standard text editor like Atom, Sublime, Notepad++, or Emacs are fine ways to edit code, an Integrated Development Environment, or IDE for short, provides some features that you might find useful. There are several good IDEs available, including Eclipse, Netbeans, and Intellij IDEa. We'll use Eclipse.

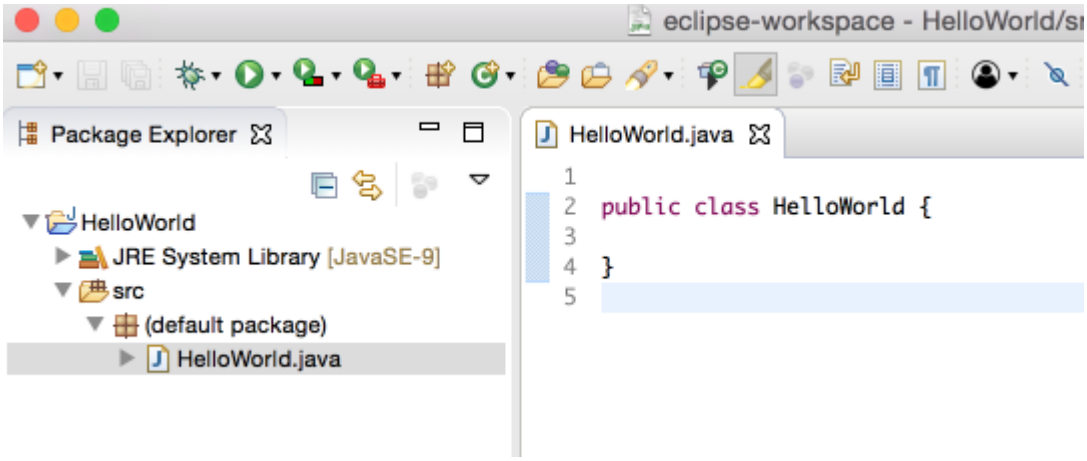
1. Download the installer from the Eclipse download page.
2. Run the intaller, and select "Eclipse IDE for Java Developers" from the menu.
3. Launch Eclipse, and select a workspace, which will be the folder where your projects with Java code will be kept.

4. Create a new Java project, using from “File->New...-> Java project”. Call the project HelloWorld, and leave the other options set to default values:



Hello World in Eclipse

You should now be at the main editor window for Eclipse. From the File menu, select New --> Class, and create a class `HelloWorld`. Your screen should now look something like this:



You now have a class called `HelloWorld`, but it cannot be run without a `main` method. Add a `main` method to print some text:

```
public class HelloWorld {
        public static void main(String[] args) {
                System.out.println("Hello, world.");
        }
}
```

Once you've added the code to Eclipse, run it using either "Run As" from the "Run" menu, or using the small green arrow icon from the top Eclipse toolbar. In the console at the bootom of the screen, you should see the text `Hello, world.`.

The class `HelloWorld` is stored in its own file, `HelloWorld.java`. Each new class you create will get its own file, and you can use the package explorer on the left to move between files.

Exercise: creating a new class

In the Eclipse IDE, create a new class `Location` with `x` and `y` instance variables, a constructor, and a toString method. Then create a Location object

from `main` and print out a Location or two to verify that you can run the program from Eclipse.

Here's the code that I wrote to test Eclipse; notice that the `Location` class is in a different file than `HelloWorld`. The sample code is not executable and you should not look at it unless you are having trouble getting Eclipse to run the code.

Eclipse test | HelloWorld.java | Location.java

```
Sample code in next two tabs.
```

Step 1: Getting the starter code

Explore the provided code for the Tetrominos user interface, in preparation for writing the game.

To get you started, and let you focus on practicing Java skills rather than writing user interface code, I have written the complete Tetrominos class and a little bit of the Board class. To use them, create a class for each using Eclipse, and cut-and-paste the code for each class into Eclipse. The code is below in the two tabs of the code window. Don't worry yet about how this code works; we'll go over it soon.

Once you have added this code to your Eclipse project, use Eclipse to run Tetrominos.java. You should see a window with a black background, containing a blue rectangle.

Tetrominos.java | Board.java

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;

import javax.swing.JFrame;
import javax.swing.Timer;

// The class that sets up the user interface
// for the Tetrominos game. You will not need to
// edit this file, but it might be worth reading through it.

class Tetrominos {

        private static void createAndShowGUI() {
                //Create and set up the window.
                JFrame frame = new JFrame("Tetrominos!");
                frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

                Board board = new Board(16, 32);

                frame.getContentPane().add(board);

                //Display the window.
                frame.pack();
                frame.setResizable(false);
                frame.setVisible(true);
```

```
                Timer timer = new Timer(300, new ActionListener() {
                        public void actionPerformed(ActionEvent evt) {

                                board.nextTurn();
                        }

                });

                frame.addKeyListener(new KeyListener() {
                        public void keyPressed(KeyEvent e) {
                                int key = e.getKeyCode();
                                //System.out.println("key pressed " + e.getKeyCode());
                                if(key == KeyEvent.VK_D) {
                                        board.rotateLeft();
                                } else if (key == KeyEvent.VK_G) {
                                        board.rotateRight();
                                } else if (key == KeyEvent.VK_S) {
                                        board.slide(-1);
                                }  else if (key == KeyEvent.VK_F) {
                                        board.slide(1);
                                }
                        }

                        @Override
                        public void keyTyped(KeyEvent e) {
                        }

                        @Override
                        public void keyReleased(KeyEvent e) {
                        }
                });

                timer.start();
        }

        public static void main(String[] args) {
                javax.swing.SwingUtilities.invokeLater(new Runnable() {
                        public void run() {
                                createAndShowGUI();
                        }
                });

        }

}
```

Step 2: The Board class

The interesting modeling of the game will be in `Board.java` and other class files. So start reading code at `Board.java`.

WE'LL COVER THE FOLLOWING

- Exercise: test user actions
- Drawing to the screen with Graphics objects

To focus on learning Java, you may reasonably ignore the Tetrominos class. It will be the file that you run to start the program, but the `Board` class can be thought of as the core of your Tetrominos program, and it's where you'll write the most code. Go to Eclipse and start reading the code now.

The keyword `extends` and some provided code in the constructor set the Board up as a user interface component that can be added to a window, and you may safely ignore this code.

Each of the other existing methods in the Board class will be called in response to certain actions, either by the user, or due to time passing. Specifically:

- `paintComponent` will be called any time you need to draw the blocks on the screen.
- `nextTurn` will be called every 300 milliseconds, when it is time to move the active piece downwards on the screen.
- `slide`, `rotateLeft`, and `rotateRight` will be called when the user presses control keys to move the game piece. Calls to `repaint()` in these methods request that `paintComponent` be called so that you can see the results of the action.

Exercise: test user actions

Add `System.out.println` calls to `nextTurn`, `slide`, `rotateLeft`, and `rotateRight`, and run the program. Observe how methods are called in response to the timer and in response to key presses

Drawing to the screen with Graphics objects

Swing drawing is done using objects of the `JComponent` class. The `extends` keyword in the definition of `Board` indicates that a Board can be used as a JComponent, and has several standard methods available to it. Once a JComponent has been added to the user interface (in this case by Tetrominos.java), the special method `paintComponent` will be called whenever the user interface needs to draw that component.

So, if you put drawing code in `paintComponent` of the `Board` class, that drawing code will be called at least once, when the main window is initially drawn. When `paintComponent` is drawn, it is passed a parameter, a reference to a Graphics object. You can draw on the component using methods of this Graphics object.

The starter code I've provided draws two rectangles:

```
public void paintComponent(Graphics g) {
        // clear the screen with black
        g.setColor(Color.black);
        g.fillRect(0, 0, getWidth(), getHeight());

        // demonstrate drawing a rectangle
        g.setColor(Color.blue);
        g.fillRect(120, 120, 42, 19);

}
```

The `setColor` method takes as a parameter a `Color` object. Several references to Color objects are available in the `Color` class, which must be imported. In Eclipse, verify that `java.awt.Color` has been imported. (`awt` is a reference to the Abstract Windowing Toolkit, which was user interface library that shipped with Java before Swing was developed.)

The `fillRect()` method has parameters that indicate the x and y coordinates of the upper left of the rectangle, and width in height. Go to Exlipse and experiment with drawing a few other rectangles. Don't forget to run the

experiment with drawing a few other rectangles. Don't forget to run the program using Tetrominos.java.

When you are done, delete all of the code in paintComponent except the first two calls that draw a black rectangle as the background for your game.

You might notice the calls to `getWidth()` and `getHeight()`. These calls are methods of the JComponent class, and therefore are available to call on the `this` object passed into `paintComponent`.

Step 3: The Block class

Add a new class to represent and draw individual square blocks in the Tetrominos game.

WE'LL COVER THE FOLLOWING

- Task: write the constructor
- Task: write the `draw` method

Each falling piece in Tetrominos is made up of four small blocks arranged in a particular way. Let's write and test the `Block` class. Create a new file in your project called `Block.java`. To get you started, here's a skeleton of the code, which you may copy in:

Block.java | BlockSolution.java

```
import java.awt.Color;
import java.awt.Graphics;

class Block {

        public int colorIndex;

        public static Color[] colors = {Color.red, Color.blue, Color.magenta,
                        Color.orange, Color.green, Color.cyan, Color.yellow};

        public Block(int colorIndex) {
        }

        public void draw(Graphics g, int scale, int x, int y) {
        }

}
```

Task: write the constructor

Blocks come in several colors. To simplify referring to the colors, we use an array of colors, defined in the array `colors`. This array is `static` because, although we will create many blocks, there will be only one array of colors,

accessible through the `Block` class.

Each block has exactly one instance variable, as we can see from the line `public int colorIndex`. If the block has a colorIndex of 2, then this would indicate that the block should be drawn using the magenta color. When a block is created using its constructor, the index of the color of the block should be passed in, and stored in the instance variable.

Write the body of the constructor, which should be only one line long. When you are done, you may check your work by looking at the second tab of the code above, `BlockSolution.java`.

Task: write the `draw` method

The Block class maintains information about the color of the block, but nothing else. Still, the Block class is a useful place to put a method that draws a block of the proper color. Write the `draw()` method, which takes as parameters an x, y coordinate in **block coordinates**, and a scaling parameter `scale`. To find the pixel coordinates, multiply `x` and `y` by `scale`, and then use the Graphics object `g` to draw the rectangle.

Finishing Tetrominos

A few hints on how to finish the Tetrominos game.

At this point, you should have a well-organized working program that lets you draw colored blocks on the screen. As this is a challenge project, the rest is up to you. I recommend breaking the project down into manageable chunks, setting some intermediate goals, and reaching those goals one at a time. There is no hurry – take plenty of time to think through each step, and test it before going on to the next item.

Here are some things you'll need to do:

1. Create the falling pieces, which are made up of multiple blocks. I wrote a class `Piece` to keep track of information for a small group of blocks.

2. Each block needs a physical location within the piece – the layout of the piece. I created a class `Location` that simply stores an x, y pair. A `Piece` then contains a list of locations.

3. Each Piece needs to be able to fall, so each piece needs to know where it is on the screen. I added an instance variable `position` of type `Location` to the Piece class. I wrote a method `translate` in the Piece class. To actually make the piece fall, I had a line of code `activePiece.translate(0, -1);` in the `nextTurn` method of the Board class. Since the `nextTurn` method is called every second or so by the Tetrominos user interface, this causes the active piece to fall downwards one step at a time.

4. The blocks should translate left and write as the user presses the `s` and `f` keys on the keyboard. When those keys are pressed, the `slide` method of Board.java is called. Finish the `slide` method.

5. Add rotation in response to key presses. This step is trickiest, and you might leave it until last, since you can write a working game without it. You'll need to figure out how locations within a piece change on rotation.

6. Check for collisions between the falling *active* piece and the bottom of the screen, and blocks that are already sitting on the bottom of the screen. When there is a collision, copy block locations from the Piece to the Board to represent the leftover blocks, and then create a new active piece.

7. Check for complete rows of blocks in in the board, and delete them.

Good luck, and happy coding!

Bonus: Tetrominos User Interface with Swing

What you need to know about the built-in Java user interface library to write Tetrominos.

WE'LL COVER THE FOLLOWING

- Starting the Swing program from main
- Creating a window
- Creating and adding a JLabel component
- Displaying the user interface
- Timer events
- Keyboard events
- Example: the Tetrominos class

User interface coding is an involved task. You will not need to write any user interface code yourself for Tetrominos, since I will provide this portion for you. However, it's nice to know what types of things you can do with a user interface library, and this section will give a very brief introduction.

There are several user interface libraries for Java. We will use Swing, which is provided with Java. The user interface library handles putting windows, buttons, menus, and other elements on the screen, and allows for user input such as mouse motion, clicking, or key presses.

Let's start with an example. First, create a new project with Eclipse. Call it whatever you like; for example, `SwingExamples`. Create a new class, called SwingTest. Then copy-paste the following code into SwingTest. (This code is based on Oracle's HelloWorldSwing example.) Do not worry too much about the details of how the code works yet.

Run the code. If everything is working properly, a small window should appear somewhere on your screen containing the words "Hello, World". You can click the close button on that window to stop the program.

```
import javax.swing.*;

public class SwingTest {
    private static void createAndShowGUI() {
        // create a new window, called a "frame"
        JFrame frame = new JFrame("Welcome to Swing");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // create a label and add it to the window
        JLabel label = new JLabel("Hello World");
        frame.getContentPane().add(label);

        //Display the window.
        frame.pack();
        frame.setVisible(true);
    }

    public static void main(String[] args) {

        javax.swing.SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                createAndShowGUI();
            }
        });
    }
}
```

Starting the Swing program from main

Like any other Java program, the code starts executing in `main`. There's a fairly complicated command in `main`. Although we will not go into the details of how this works, the effect of the call to `invokeLater` is to call the method `createAndShowGUI()`. No other code should go in the main method; we may treat `createAndShowGui` as the entry point for the program.

The reasons `invokeLater` is needed are somewhat arcane, and have to do with the interaction between Swing and the operating system. The operating system captures events like key presses and mouse clicks from the user, and must pass those events along to a Swing program for handling. The call to the `invokeLater` method initializes Swing in such a way that it can properly receive and handle those events. To create a new Swing program, you can copy this `main` method in, without worrying about the details too much.

Creating a window

The Swing library is built around the idea of *components*: items like windows,

labels, buttons, or drawing areas are created by created appropiate objects. The first line of code executed in `createAndShowGUI` creates a JFrame object representing a window on the screen. A window often has a title, shown at the top, and the constructor sets the title of the new window to `"Welcome to Swing"`.

```
// create a new window, called a "frame"
JFrame frame = new JFrame("Welcome to Swing");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Methods on Swing objects can be used to change their behavior or properties. The call to `setDefaultCloseOperation` in this case indicates that the program should exit when the window is closed. (If the program allowed mutliple windows to be open, you might want some other behavior, and allow individual windows to be closed without quitting the program.)

Creating and adding a JLabel component

Components can act as containers. A window might contain a collection of buttons, some of which might contain text labels. In Swing, there are methods to add components to other components. For example:

```
// create a label and add it to the window
JLabel label = new JLabel("Hello World");
frame.getContentPane().add(label);
```

Although the `Frame` is a window, a window has different parts, including the title of the window, scrollbars, etc. `frame.getContentPane()` fetches the main region of the window where custom components might be added. Then `add` is called on that region to add the label to it.

Notice the chaining of method calls: `frame.getContentPane().add(label);` should be read left-to-right, recognizing that the result of `getContentPane` is itself a reference to an object upon which methods may be called.

Displaying the user interface

The size of the window should be determined by the components it contains. `frame.pack()` resizes the window appropriately so that the window is just big enough to hold the label.

It takes time to draw user interface components on the screen, and it is distracting if the user sees individual elements being added to a window one at a time, or if the window is suddenly resized on the screen without warning. By default, the window is invisible, so that initial set-up can be completed. Once ready, you can call `frame.setVisible(true)` to display the window.

Timer events

It can be useful for a program to take a certain action every once in a while. For example, in a game of Tetris, blocks must move down a row every 300 milliseconds or so. Swing provides the capabilities to do this, using a `Timer` object. If you add the following code to the end of `createAndShowGUI` after the frame has been displayed, the `println` method will be called every 300 milliseconds.

```
        Timer timer = new Timer(300, new ActionListener() {
                        public void actionPerformed(ActionEvent evt) {
                                System.out.println("Time is up!");
                        }

                });
```

The structure of the code above relies on some Java concepts that have not been introduced in these lessons. For now, it is enough to experiment with the results.

Keyboard events

It is also useful if a program can execute some piece of code every time a particular key is pressed. In the Tetrominos game, pressing the `s` key should trigger a method to be called that moves the piece to the left. Take a look at the provided code for the Tetrominos user interface in the next section to see how keys can be captured using a KeyListener object.

Example: the Tetrominos class

The Tetrominos class is the main entry point for the Tetrominos program, and you should already have it in Eclipse. Let's take a high-level look at it:

`Tetrominos` opens a window with `new JFrame()` and displays it.

We use `frame.getContentPane().add(board);` to add `board` as a component to the window. Primarily, this means that the Board class can use graphics commands to draw onto the window.

Then a `Timer` is created that will call the method `nextTurn()` of the board class every 300 milliseconds. You'll write the body of the `nextTurn()` method later on.

Finally, keyboard input handling is set up. When the user presses one of the keys `s`, `d`, `f`, or `g`, certain methods of the Board class will be called.